Lecture Notes: SHAP Values

CIC1205 - Machine Learning

Eduardo Bezerra

May 16, 2026

0 Contents

Before You Read **2**

Learning Objectives **2**

1 Introduction **3**
1.1 The Interpretability Problem . . . 3
1.2 What Are SHAP Values? . . . 3
1.3 Game-Theoretic Foundations: Shapley Values . . . 4

2 Steps to Compute SHAP Values **5**
2.1 Step 1: Choose a Background Dataset . . . 5
2.2 Step 2: Specify the Model and the Instance to Explain . . . 6
2.3 Step 3: Compute the SHAP Baseline Value ϕ_0 . . . 7
2.4 Step 4: Compute Each SHAP Value . . . 7

3 Understanding Marginal Contributions **10**
3.1 The Meaning of $h(x_S)$ . . . 11
3.2 Context-Dependence: A Nonlinear Example . . . 11

4 Full Numerical Example **13**
4.1 Setup . . . 13
4.2 Step 3: Computing the Baseline ϕ_0 . . . 13
4.3 Step 4: Computing ϕ_{x_1} . . . 13
4.4 Step 4: Computing ϕ_{x_2} . . . 14
4.5 Verification: Decomposition Property . . . 15

5 Key Properties of SHAP Values **16**
5.1 Axiomatic Justification . . . 16
5.2 Computational Complexity . . . 16

6 SHAP in Practice: The SHAPE Library **17**
6.1 What Output Are We Explaining? . . . 17
6.2 Overview . . . 17
6.3 Choosing the Right Explainer . . . 18
6.4 Computing SHAP Values . . . 18
6.5 Visualization . . . 19

7 Interpreting SHAP Values **22**
7.1 Local vs. Global Explanations . . . 23
7.2 Reading a Waterfall Plot . . . 23
7.3 Reading a Beeswarm Plot . . . 23

8 Final Remarks **24**
8.1 Summary of Key Concepts . . . 24
8.2 Common Pitfalls . . . 25
8.3 Connection to Other Interpretability Methods . . . 26

0 Before You Read

These notes assume that you are already familiar with the following topics. If any of them feel unclear, review the corresponding previous notes before studying SHAP values.

- Supervised machine learning: the concepts of training, features, labels, and prediction functions.
- Basic probability and expected values.
- At least one trained machine learning model (e.g., a decision tree, random forest, or neural network) that you can apply to produce predictions.
- Set notation: subsets, unions, and the notion of a power set.
- Summation notation and weighted averages.

0 Learning Objectives

After studying these notes, you should be able to:

- explain what SHAP values are and why they are useful for interpreting machine learning models;
- describe the connection between SHAP values and Shapley values from cooperative game theory;
- define and compute the SHAP baseline value ϕ_0 from a background dataset;
- compute marginal contributions Δ_i^S and derive SHAP values ϕ_i by hand for a small example;
- verify the SHAP decomposition property $h(x) = \phi_0 + \sum_i \phi_i$;
- apply the SHAP Python library to explain predictions of a trained model;
- interpret summary plots, waterfall plots, and force plots produced by the SHAP library.

1 Introduction

1.1 The Interpretability Problem

Modern machine learning models such as gradient-boosted trees and deep neural networks are highly accurate on many tasks. However, their internal computations are complex and opaque: given an input, the model produces a prediction, but it is not immediately obvious *why* it made that prediction. This lack of transparency creates significant practical challenges.

In high-stakes settings such as medicine, finance, or law, stakeholders need to understand *which features drove a particular prediction*. A doctor relying on a diagnostic model, or a loan officer using a credit-scoring system, may be required by law or professional ethics to justify a decision. A data scientist needs to verify that a model is using sensible signals, not spurious correlations. These needs motivate the field of *explainable artificial intelligence (XAI)*.

Interpretability

A machine learning model is *interpretable* to the extent that a human can understand the reasons behind its predictions. Local interpretability explains a single prediction for one instance, while global interpretability explains the overall behavior of the model across all inputs.

1.2 What Are SHAP Values?

SHAP (*SHapley Additive exPlanations*) is a framework for explaining the output of any machine learning model [Lundberg and Lee, 2017]. Given a trained model h and a specific input instance x, SHAP assigns to each feature x_i a numerical value ϕ_i that quantifies how much that feature contributed to the prediction $h(x)$, relative to a reference baseline.

More precisely, SHAP can be applied, in principle, to any model that can be evaluated as a prediction function. However, the computational method and the meaning of the resulting SHAP values depend on the model family and on the output scale being explained, such as a probability, a log-odds score, or a raw model score. We will return to this point in Section 6.1.

SHAP Values

SHAP values $\phi_1, \phi_2, \ldots, \phi_n$ are feature attribution scores that satisfy the so called *decomposition property*:

$$h(x) = \phi_0 + \sum_{i=1}^{n} \phi_i,$$

where ϕ_0 is a baseline value representing the model's expected output in the absence of any specific feature information, and each ϕ_i measures the contribution of feature i to the deviation of $h(x)$ from that baseline. SHAP values can be positive (the feature pushes the selected output above baseline) or negative (the feature pulls the selected output below baseline).

The key appeal of SHAP values is that they apply to *any* model (a neural network, a random forest, a linear regression) and provide a *consistent*, *locally accurate* explanation for each individual prediction.

SHAP Explains the Model, Not Necessarily Reality

A SHAP value tells us how much a feature contributed to the *model's* prediction. It does not prove that the feature caused the real-world outcome. If the model has learned a spurious correlation, SHAP will faithfully explain that spurious behavior. In this sense, SHAP is a tool for auditing a model's reasoning, not a guarantee that the reasoning is scientifically, causally, or ethically valid. This distinction is especially important in high-stakes domains such as medicine, credit scoring, and criminal justice.

1.3 Game-Theoretic Foundations: Shapley Values

SHAP values are grounded in **cooperative game theory**[1], specifically in the concept of Shapley values introduced by Lloyd Shapley [Shapley, 1953]. The Shapley value answers a fundamental fairness question: given a group of players who cooperate to produce a collective payout, how should that payout be fairly distributed among the players according to each player's contribution? In machine learning, SHAP adapts this idea by treating the features of an instance as the players and the model prediction as the payout to be explained.

In the machine learning context, the analogy is:

- **Players** $\leftrightarrow$ features $x_1, x_2, \ldots, x_n$.
- **Payout** $\leftrightarrow$ the model's prediction $h(x)$.
- **Coalition** $\leftrightarrow$ a subset $S \subseteq \{1, \ldots, n\}$ of features that are "known".
- **Fair distribution** $\leftrightarrow$ the SHAP values ϕ_i that sum to $h(x) - \phi_0$.

Shapley proved that there is a *unique* allocation satisfying four natural axioms (efficiency, symmetry, linearity (additivity), and dummy player) and that allocation is precisely the weighted average of each player's marginal contribution over all possible coalitions.

Shapley Axioms vs. SHAP Properties

The original cooperative game theory formulation characterizes Shapley values through four classical axioms: *efficiency* (all payout is distributed), *symmetry* (equivalent players are treated equally), *linearity* (contributions are additive across games), and *dummy player* (a player who contributes nothing receives nothing). The SHAP framework adapts this idea to machine learning and emphasizes a related but differently labeled set of properties: *local accuracy*, *missingness*, *consistency*, and *symmetry*. Although these lists are closely related, they come from slightly

[1] Cooperative game theory studies situations in which multiple agents, called *players*, can form coalitions and cooperate to obtain a collective payoff. A central question is how to distribute that payoff among the players in a way that reflects their individual contributions. In the SHAP setting, the "players" are the input features, and the "payoff" is the model prediction.

different formulations; the original game-theoretic setting and the SHAP framework for additive feature attribution methods. The key practical consequence is the same: SHAP values are the *unique* additive attribution method satisfying all of these desirable properties simultaneously.

Common Misconception

SHAP values do not measure whether a feature is globally important to the model. They measure *how much a specific feature value contributed to a specific prediction*. The same feature may have a large positive SHAP value for one instance and a large negative SHAP value for another. Global feature importance can be derived by aggregating SHAP values across many instances, but a single SHAP value is always a local quantity.

Entry Ticket 1

Before reading further, think about the following questions.

1. You are given a model that predicts house prices. For a specific house, the model predicts \$450,000. Can you immediately tell which features (e.g., square footage, number of rooms, location) contributed the most to this price? Why or why not?
2. In a group project where four students receive the same grade, it seems unfair if one student did all the work. What information would you need to distribute the grade fairly? How does this relate to the idea of SHAP values?
3. Name one practical reason why understanding individual model predictions might be important, beyond curiosity.

2 Steps to Compute SHAP Values

Computing SHAP values for an instance x involves four main steps: choosing a background dataset, specifying the model and instance, computing the baseline, and computing the weighted marginal contributions of each feature. Each step is described in detail below.

2.1 Step 1: Choose a Background Dataset

SHAP requires a reference distribution to simulate "missing" features. This reference is called the *background dataset*.

Background Dataset

The *background dataset* $D \subseteq \mathbb{R}^n$ is a set of representative input examples used to estimate the model's expected output when some or all features of a given instance are unknown:

$$D = \{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}.$$

It is typically a sample from the training data and should reflect the marginal distribution of the input features.

A practical question when setting up SHAP is what it means for a feature to be "missing" or "unknown". A machine learning model cannot be evaluated on incomplete inputs: it always requires all features to produce a prediction. SHAP resolves this by substituting missing feature values with values drawn from the background dataset, effectively marginalizing over the unknown features. The background dataset thus defines the reference against which each feature's contribution is measured.

The Background Dataset Defines the Reference Point

SHAP values are not absolute quantities; they are measured *relative* to a reference population represented by the background dataset. To see why this matters, consider a model that predicts income. If the background dataset consists mostly of high-income individuals, then ϕ_0 will be high, and SHAP values will explain deviations from a high-income baseline. If the background dataset is instead a representative population sample, ϕ_0 will reflect an average prediction for the general population. Consequently, the same instance can receive different SHAP values under different background datasets. Always use a background that reflects the intended input distribution for the explanation.

Choosing the Background Dataset in Practice

In practice, the background dataset is usually a random sample of 50–200 instances reused from the training set, although a separate representative reference dataset may also be used when available. Using the entire training set is computationally expensive and rarely necessary.

This reuse of training examples is not data leakage in the usual evaluation sense, because SHAP is not training the model or estimating its generalization performance. SHAP is explaining an already trained model relative to a chosen reference population.

In some applications, the background dataset can be replaced by a single reference instance, such as an all-zero vector. This means that SHAP will explain predictions relative to that specific reference point, rather than relative to an empirical sample of examples.

This choice should be explicit and justified. Changing the reference point changes the baseline value ϕ_0 and, consequently, the SHAP values assigned to the features.

2.2 Step 2: Specify the Model and the Instance to Explain

Let $h : \mathbb{R}^n \to \mathbb{R}$ be the trained model's prediction function (what we have elsewhere called the hypothesis). Let $x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ be the specific input instance whose prediction $h(x)$ we wish to explain.

The goal of the remaining steps is to decompose $h(x)$ into a sum of contributions, one per feature, plus a baseline term. The features of x are the "players" in the cooperative game, and $h(x)$ is the "payout" to be fairly distributed.

2.3 Step 3: Compute the SHAP Baseline Value ϕ_0

The baseline value ϕ_0 represents the model's expected prediction when *no feature information* about the specific instance x is available. More precisely, it is the average prediction the model would make if we had to rely solely on the background distribution, without any knowledge of the specific values of x. This does not mean that the model is literally evaluated without features; rather, we average the model's predictions over all instances in the background dataset, and use this average as the anchor from which feature contributions are measured.

SHAP Baseline

The SHAP baseline is defined as the expected model output over the background dataset:

$$\phi_0 = \mathbb{E}_{x' \sim D}[h(x')] = \frac{1}{m} \sum_{j=1}^{m} h(x^{(j)}).$$

Example. Suppose the model is $h(x_1, x_2) = 3x_1 + 5x_2$ and the background dataset is:

$$D = \{(0,0),\ (0,1),\ (1,0),\ (1,1)\}.$$

Then:

$$\phi_0 = \frac{1}{4}\big[h(0,0) + h(0,1) + h(1,0) + h(1,1)\big] = \frac{1}{4}\big[0 + 5 + 3 + 8\big] = 4.$$

2.4 Step 4: Compute Each SHAP Value

We now compute one SHAP value ϕ_i for each feature i. The idea is to measure how much the prediction changes when feature i is added to different sets of already-known features.

Let

$$F = \{1, 2, \ldots, n\}$$

be the set of feature indices. For a given feature i, we consider all subsets

$$S \subseteq F \setminus \{i\}.$$

Each subset S represents a coalition of features that are already known before feature i is added.

Model Output When Only Some Features Are Known

The model h always receives a complete input vector. Therefore, before defining marginal contributions, we need a notation for the model's expected output when only some features of the target instance are known.

Let the target instance be

$$x = (x_1, x_2, \ldots, x_n),$$

and let $S \subseteq F$ be a subset of feature indices.

We write $h(x_S)$ for the expected model output when the features in S are fixed to their values in the target instance x, and all features outside S are filled in using values from the background dataset.

Thus, $h(x_S)$ does *not* mean that the model is evaluated using only the features in S. The model always receives a complete input vector.

More explicitly, let the target instance be

$$x = (x_1, x_2, \ldots, x_n),$$

and let a background instance be

$$x' = (x'_1, x'_2, \ldots, x'_n).$$

For a subset $S \subseteq F$, define the completed input vector

$$z(S, x, x') = (z_1, z_2, \ldots, z_n)$$

by

$$z_k = \begin{cases} x_k, & \text{if } k \in S, \\ x'_k, & \text{if } k \notin S. \end{cases}$$

Thus, features in S come from the instance being explained, while features outside S come from a background example.

Under the interventional definition used in these notes,

$$h(x_S) = \mathbb{E}_{x' \sim D}\left[h\big(z(S, x, x')\big)\right].$$

In practice, if the background dataset is

$$D = \{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\},$$

we approximate this expectation by the empirical average

$$h(x_S) \approx \frac{1}{m} \sum_{j=1}^{m} h\big(z(S, x, x^{(j)})\big).$$

When $S = \emptyset$, no feature from the target instance is known. Therefore,

$$h(x_\emptyset) = \phi_0.$$

When $S = F$, all features from the target instance are known. Therefore,

$$h(x_F) = h(x).$$

Marginal Contribution

Now consider feature i. Suppose that the features in S are already known. The marginal contribution of feature i to this coalition is the change in expected model output when we additionally reveal feature i:

$$\Delta_i^S = h(x_{S \cup \{i\}}) - h(x_S).$$

The first term, $h(x_{S \cup \{i\}})$, is the expected prediction when the features in S and feature i are fixed to their values in the target instance. The second term, $h(x_S)$, is the expected prediction when only the features in S are fixed.

Thus, Δ_i^S answers the question:

> Given that we already know the features in S, how much does the expected prediction change when we also learn feature i?

A Small Example of the Notation

Suppose there are three features and

$$F = \{1, 2, 3\}.$$

Let the target instance be

$$x = (x_1, x_2, x_3),$$

and let a background instance be

$$x' = (x'_1, x'_2, x'_3).$$

If $S = \{2\}$, then only feature 2 is fixed to its value in the target instance. The completed input is

$$z(\{2\}, x, x') = (x'_1, x_2, x'_3).$$

Therefore,

$$h(x_{\{2\}}) = \mathbb{E}_{x' \sim D}\left[h(x'_1, x_2, x'_3)\right].$$

If we are computing the SHAP value for feature 1 and add feature 1 to this coalition, then

$$S \cup \{1\} = \{1, 2\}.$$

The completed input becomes

$$z(\{1, 2\}, x, x') = (x_1, x_2, x'_3),$$

and

$$h(x_{\{1,2\}}) = \mathbb{E}_{x' \sim D}\left[h(x_1, x_2, x'_3)\right].$$

Thus, the marginal contribution of feature 1 when feature 2 is already known is

$$\Delta_1^{\{2\}} = h(x_{\{1,2\}}) - h(x_{\{2\}}).$$

Subset Weight

Not all subsets S should contribute equally. SHAP weights each marginal contribution according to how often that coalition appears before feature i across all possible orderings of the features:

$$w_S = \frac{|S|!\,(|F| - |S| - 1)!}{|F|!}.$$

This weight is the fraction of all feature orderings in which exactly the features in S appear before feature i.

For example, if $|F| = 3$ and we are computing the SHAP value for x_1, then the possible subsets are:

$$\emptyset, \quad \{2\}, \quad \{3\}, \quad \{2, 3\}.$$

Their weights are:

$$w_\emptyset = \frac{0!2!}{3!} = \frac{1}{3},$$

$$w_{\{2\}} = \frac{1!1!}{3!} = \frac{1}{6},$$
$$w_{\{3\}} = \frac{1!1!}{3!} = \frac{1}{6},$$
$$w_{\{2,3\}} = \frac{2!0!}{3!} = \frac{1}{3}.$$

These weights sum to 1.

SHAP Value Formula

The SHAP value for feature i is the weighted average of all its marginal contributions:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} w_S \cdot \Delta_i^S.$$

In expanded form:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[h(x_{S \cup \{i\}}) - h(x_S) \right].$$

This formula says: compute the contribution of feature i in every possible context S, then average those contributions using the Shapley weights.

Why Feature Importance Is Context-Dependent

The same feature may be more or less informative depending on what other features are already known. For example, knowing a patient's body weight is highly informative when no other health indicators are available, but may become redundant once we already know their BMI. Δ_i^S captures exactly this context dependence: it measures the contribution of feature i in the specific context where the features in S are already known. SHAP values summarize this contribution fairly across all possible contexts by taking a weighted average over all coalitions S.

Entry Ticket 2

Consider a model with three features $\{x_1, x_2, x_3\}$ and you are computing the SHAP value for x_1.

1. List all possible subsets $S \subseteq \{x_2, x_3\}$ (i.e., all subsets of $F \setminus \{1\}$).
2. For each subset, write out the expression for Δ_1^S in terms of $h(\cdot)$ evaluations.
3. Compute the weight w_S for each subset using the formula above, with $|F| = 3$.
4. Verify that the weights sum to 1.

3 Understanding Marginal Contributions

Before computing a full numerical example, it is worth building intuition about what Δ_i^S represents and why the weighted average of these quantities gives a "fair" attribution.

3.1 The Meaning of $h(x_S)$

When we write $h(x_S)$, we mean: the model's expected output when only the features indexed by S are fixed to their values in the target instance x, and all other features are replaced with values from the background dataset. This is not a partial model evaluation; the model always receives a complete input. Instead, we simulate the absence of features by substituting them with background values.

Concretely, if $n = 2$, $S = \{1\}$, and the background dataset has m instances:

$$h(x_{\{1\}}) = \frac{1}{m} \sum_{j=1}^{m} h(x_1,\ x_2^{(j)}),$$

where x_1 is the value of the first feature in the target instance, and $x_2^{(j)}$ runs over the second-feature values in the background. We fix what we "know" (feature 1) and average over what we "do not know" (feature 2).

3.2 Context-Dependence: A Nonlinear Example

To see why marginal contributions depend on context, consider the model $h(x_1, x_2) = x_1 \cdot x_2$ with the background dataset $D = \{(0,0), (0,1), (1,0), (1,1)\}$ and target instance $x = (1,1)$.

Context 1: No features known ($S = \emptyset$).

$$h(x_\emptyset) = \frac{1}{4}[0 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 + 1 \cdot 1] = 0.25.$$

$$h(x_{\{1\}}) = \frac{1}{2}[1 \cdot 0 + 1 \cdot 1] = 0.5.$$

$$\Delta_1^\emptyset = 0.5 - 0.25 = 0.25.$$

Context 2: Feature 2 already known ($S = \{2\}$).

$$h(x_{\{2\}}) = \frac{1}{2}[0 \cdot 1 + 1 \cdot 1] = 0.5.$$

$$h(x_{\{1,2\}}) = h(1,1) = 1.$$

$$\Delta_1^{\{2\}} = 1 - 0.5 = 0.5.$$

The same feature $x_1 = 1$ contributes 0.25 when x_2 is unknown, but 0.5 when $x_2 = 1$ is already known. This difference arises because h is multiplicative: knowing $x_2 = 1$ amplifies the contribution of x_1. SHAP accounts for both contexts by taking a weighted average:

$$\phi_1 = w_\emptyset \cdot \Delta_1^\emptyset + w_{\{2\}} \cdot \Delta_1^{\{2\}} = \frac{1}{2}(0.25) + \frac{1}{2}(0.5) = 0.375.$$

How SHAP Handles Interaction Effects

Notice that neither ϕ_1 nor ϕ_2 alone equals $h(1,1) = 1$. The model output $h(x_1, x_2) = x_1 \cdot x_2$ only becomes non-trivial when *both* features are known: the entire interaction effect only materializes when the two features are observed together. SHAP handles this by averaging over all orderings of the features. When x_1 is revealed first (before x_2 is known), it gets a small marginal credit; when x_1 is revealed second (after x_2 is already known), it gets a larger credit. The final SHAP value $\phi_1 = 0.375$ is the average of these two situations. By symmetry, $\phi_2 = 0.375$ as well. This is what it means for SHAP to distribute the interaction effect *fairly*: neither feature takes full credit for an outcome that required both of them.

Additivity of SHAP Values

By construction, SHAP values satisfy the decomposition property: for any instance x and any trained model h,

$$h(x) = \phi_0 + \sum_{i=1}^{n} \phi_i.$$

This means the prediction is *exactly* the sum of the baseline and the SHAP values of all features. No prediction is "left unexplained". This additivity property is what makes SHAP values a complete local explanation.
Note that in numerical implementations or when using approximate explainers, this equality should hold up to small numerical or sampling error. A practical sanity check, verifying that $\phi_0 + \sum_i \phi_i \approx h(x)$, is always recommended before trusting an explanation.

Figure 1 illustrates the SHAP decomposition as a horizontal bar chart. In the schematic figures in these notes, blue is used for positive contributions and red for negative contributions. Note that the SHAP Python library may use a different color convention in its own plots (typically red/pink for features that increase the output and blue for features that decrease it); always interpret the direction of the bar or axis, not only the color.

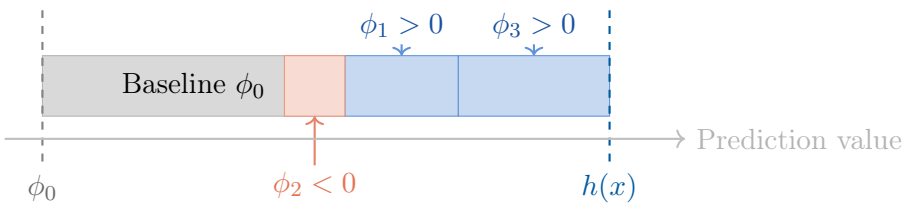

Figure 1: Schematic SHAP decomposition. The baseline ϕ_0 is the model's average prediction over the background dataset. Each feature's SHAP value shifts the selected model output up (positive, blue in these figures) or down (negative, red in these figures). The sum of all shifts plus the baseline equals the actual prediction $h(x)$.

Entry Ticket 3

Consider the multiplicative model $h(x_1, x_2) = x_1 \cdot x_2$ with background dataset $D = \{(0,0), (0,1), (1,0), (1,1)\}$.

1. Compute ϕ_0 for this model and background.
2. Using the computations above for $\phi_1 = 0.375$, compute ϕ_2 by symmetry (or by repeating the calculation for x_2).
3. Verify that $\phi_0 + \phi_1 + \phi_2 = h(1,1)$.
4. What does it mean, intuitively, that neither ϕ_1 nor ϕ_2 equals $h(1,1)/2 = 0.5$? (*Hint: think about what happens when only one feature is known.*)

4 Full Numerical Example

We now work through a complete SHAP computation for a simple linear model, where the results can be verified analytically. This example uses all four steps described in Section 2.

4.1 Setup

Model: $h(x_1, x_2) = 3x_1 + 5x_2$.

Instance to explain: $x = (1,1)$, so $h(1,1) = 3 + 5 = 8$.

Background dataset:

$$D = \{(0,0),\ (0,1),\ (1,0),\ (1,1)\}.$$

4.2 Step 3: Computing the Baseline ϕ_0

Applying the model to each background instance:

$x^{(j)}$	$h(x^{(j)})$	
$(0,0)$	0	
$(0,1)$	5	
$(1,0)$	3	
$(1,1)$	8	
Average	4	$= \phi_0$

$$\phi_0 = \frac{1}{4}(0 + 5 + 3 + 8) = 4.$$

The model predicts 4 on average over the background. Our target instance yields $h(1,1) = 8$, which is 4 units above baseline. SHAP will explain how $x_1 = 1$ and $x_2 = 1$ jointly account for this surplus.

4.3 Step 4: Computing ϕ_{x_1}

There are two possible coalitions $S \subseteq \{x_2\}$: $S = \emptyset$ and $S = \{x_2\}$.

Subset weights. With $n = 2$ features:

$$w_\emptyset = \frac{0! \cdot (2-0-1)!}{2!} = \frac{1 \cdot 1}{2} = \frac{1}{2}, \qquad w_{\{x_2\}} = \frac{1! \cdot (2-1-1)!}{2!} = \frac{1 \cdot 1}{2} = \frac{1}{2}.$$

Marginal contribution with $S = \emptyset$. We must compute $h(x_\emptyset)$ and $h(x_{\{x_1\}})$. With $S = \emptyset$, no features of x are fixed, so we average over all background instances:

$$h(x_\emptyset) = \phi_0 = 4.$$

For $h(x_{\{x_1\}})$, we fix $x_1 = 1$ and average over the background values of x_2:

$$h(x_{\{x_1\}}) = \frac{1}{2}\big[h(1,0) + h(1,1)\big] = \frac{1}{2}(3+8) = 5.5.$$

$$\Delta_{x_1}^{\emptyset} = 5.5 - 4 = 1.5.$$

Marginal contribution with $S = \{x_2\}$. We fix $x_2 = 1$ and average over background values of x_1:

$$h(x_{\{x_2\}}) = \frac{1}{2}\big[h(0,1) + h(1,1)\big] = \frac{1}{2}(5+8) = 6.5.$$

We fix both $x_1 = 1$ and $x_2 = 1$:

$$h(x_{\{x_1,x_2\}}) = h(1,1) = 8.$$

$$\Delta_{x_1}^{\{x_2\}} = 8 - 6.5 = 1.5.$$

SHAP value for x_1.

$$\phi_{x_1} = w_\emptyset \cdot \Delta_{x_1}^{\emptyset} + w_{\{x_2\}} \cdot \Delta_{x_1}^{\{x_2\}} = \frac{1}{2}(1.5) + \frac{1}{2}(1.5) = 1.5.$$

4.4 Step 4: Computing ϕ_{x_2}

Marginal contribution with $S = \emptyset$.

$$h(x_{\{x_2\}}) = \frac{1}{2}\big[h(0,1) + h(1,1)\big] = 6.5.$$

$$\Delta_{x_2}^{\emptyset} = 6.5 - 4 = 2.5.$$

Marginal contribution with $S = \{x_1\}$.

$$h(x_{\{x_1\}}) = 5.5 \quad \text{(computed above)}.$$

$$h(x_{\{x_1,x_2\}}) = h(1,1) = 8.$$

$$\Delta_{x_2}^{\{x_1\}} = 8 - 5.5 = 2.5.$$

SHAP value for x_2.

$$\phi_{x_2} = \frac{1}{2}(2.5) + \frac{1}{2}(2.5) = 2.5.$$

4.5 Verification: Decomposition Property

$$\phi_0 + \phi_{x_1} + \phi_{x_2} = 4 + 1.5 + 2.5 = 8 = h(1,1). \quad \checkmark$$

The prediction is exactly accounted for: the baseline contributes 4, feature x_1 contributes an additional 1.5, and feature x_2 contributes an additional 2.5. This result is also intuitive: in a linear model $h = 3x_1 + 5x_2$, the feature with the larger coefficient (x_2, coefficient 5) receives the larger SHAP value.

Figure 2 shows the SHAP decomposition for this example as a waterfall-style bar chart.

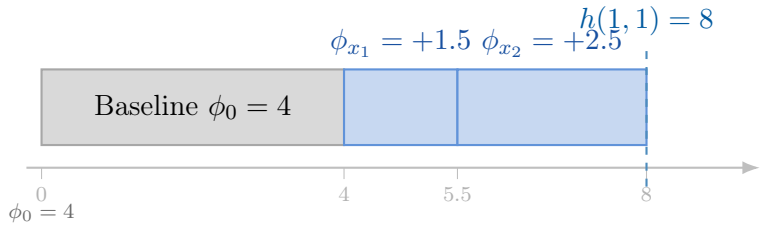

Figure 2: SHAP decomposition for the instance $x = (1,1)$ with model $h(x_1, x_2) = 3x_1 + 5x_2$ and background dataset $D = \{(0,0),(0,1),(1,0),(1,1)\}$. The baseline is 4. Feature x_1 contributes $+1.5$ and feature x_2 contributes $+2.5$, summing exactly to the prediction of 8.

Linear Models and SHAP Values

For a linear model $h(x) = \sum_{i=1}^{n} \theta_i x_i + b$, and under the interventional definition used in these notes, the SHAP value of feature i simplifies to:

$$\phi_i = \theta_i (x_i - \mathbb{E}_{x' \sim D}[x'_i]).$$

The contribution of each feature is its coefficient multiplied by how much its value deviates from the background mean. A feature whose value equals the background mean contributes zero, and a feature that deviates strongly from the mean in the direction of its coefficient contributes significantly.
Note that this simplified formula assumes the features are treated independently when filling in missing values. When features are strongly correlated and a conditional SHAP definition is used instead, attribution may be distributed differently across correlated features. For nonlinear models, SHAP values generalize this idea to all possible coalitions.

Entry Ticket 4

Consider the model $h(x_1, x_2, x_3) = 2x_1 + 4x_2 - x_3$ with background dataset:

$$D = \{(0,0,0),\ (1,0,0),\ (0,1,0),\ (0,0,1),\ (1,1,1)\}.$$

For the instance $x = (1,1,0)$:

1. Compute $h(1,1,0)$ and ϕ_0.
2. Using the linear model formula above, compute ϕ_1, ϕ_2, and ϕ_3.
3. Verify the decomposition property.
4. Which feature contributes the most to this prediction relative to baseline?

Does the sign of ϕ_3 make sense? Explain.

5 Key Properties of SHAP Values

5.1 Axiomatic Justification

The SHAP framework satisfies four desirable properties that, taken together, uniquely characterize SHAP values among all additive feature attribution methods:

- **Local accuracy (efficiency).** The SHAP values sum exactly to the model's prediction minus the baseline: $h(x) = \phi_0 + \sum_{i=1}^{n} \phi_i$. The prediction is fully explained; nothing is left unaccounted for.
- **Missingness.** If a feature has no effect on the model output (i.e., the model output does not change regardless of that feature's value), its SHAP value is zero. Irrelevant features receive zero credit.
- **Consistency.** If a model starts relying more on a feature, the attribution assigned to that feature should not go down.

  More precisely, compare two models, h and g. For every subset of already-known features S, compute the marginal contribution of feature i:

  $$\Delta_i^S = h(x_{S\cup\{i\}}) - h(x_S).$$

  If this contribution is never smaller in model g than in model h, then the SHAP value of feature i in model g should not be smaller than its SHAP value in model h.
- **Symmetry.** If two features are interchangeable (i.e., swapping their values in any coalition leaves the model output unchanged), they receive the same SHAP value. Equivalent features are treated equally.

These four properties, which are the SHAP counterparts of the classical Shapley axioms discussed in Section 1.3, ensure that SHAP values are a principled and interpretable attribution method. No other additive feature attribution method satisfies all four simultaneously.

5.2 Computational Complexity

Exact SHAP computation requires evaluating $h(x_S)$ for every subset $S \subseteq F$. The number of subsets is 2^n, so exact computation is exponential in the number of features. For a model with 20 features, this is over one million evaluations; for 50 features, it is over one quadrillion.

Computational Challenge

Exact SHAP computation has time complexity $\mathcal{O}(2^n)$ in the number of features n. For practical use with large numbers of features, approximation methods are essential. The SHAP Python library provides several such approximations, each tailored to specific model families.

The SHAP library addresses this challenge through model-specific algorithms that exploit the structure of particular model classes (e.g., decision trees) to compute exact or highly accurate approximate SHAP values in polynomial time.

Common Misconception

Approximating SHAP values does not mean getting "wrong" explanations. Methods like `TreeExplainer` can compute SHAP values efficiently and exactly for many tree-based models by exploiting the tree structure, under the feature-dependence assumptions and output scale used by the explainer. `KernelExplainer` provides model-agnostic approximations that converge to the true Shapley values as the number of samples increases. The approximation is in the computational method, not in the conceptual definition of the explanation.

6 SHAP in Practice: The SHAPE Library

6.1 What Output Are We Explaining?

Before interpreting SHAP values, we must identify precisely which scalar quantity is being decomposed. SHAP values always explain *a specific model output*, and that output can take different forms depending on the model and the configuration of the explainer:

- For a **regression model**, the output being explained is typically the predicted value $\hat{y}$.
- For a **binary classifier**, the output may be the raw model score, the log-odds, or the predicted probability, depending on the model family and the explainer configuration.
- For a **multi-class classifier**, SHAP values are usually computed separately for each class.

This distinction matters for interpretation. A positive SHAP value for feature i means *"this feature increases the selected output"*, not necessarily *"this feature increases the probability of a positive outcome"*, unless the selected output is indeed that probability. Before analyzing SHAP values, always verify what output scale is being explained.

6.2 Overview

The `shap` Python library [Lundberg and Lee, 2017] implements efficient approximations of SHAP values for a wide range of model types. It provides both the computation of SHAP values and a suite of visualization tools for interpreting them.

The general workflow is:

1. Train a machine learning model (any framework: scikit-learn, XGBoost, PyTorch, etc.).
2. Install the library: `pip install shap`.
3. Select and instantiate the appropriate `Explainer` for your model type.
4. Call the explainer on a set of instances to obtain SHAP values.

5. Visualize and interpret the results.

6.3 Choosing the Right Explainer

The `shap` library offers several explainers, each designed for a specific family of models:

Explainer	Applicable models	Notes
`TreeExplainer`	Tree-based models (XGBoost, LightGBM, Random Forest, scikit-learn trees)	Exact SHAP values for supported models; very efficient
`LinearExplainer`	Linear models (linear/logistic regression)	Exact for linear models under the interventional definition
`DeepExplainer`	Deep learning models (PyTorch, TensorFlow)	Approximate; uses back-propagation
`KernelExplainer`	Any model (model-agnostic)	Approximation via weighted regression; slowest but most general
`Explainer`	Most common models (auto-selects)	Recommended starting point

Which explainer to use. Start with `shap.Explainer`, which automatically selects the most appropriate algorithm based on the model type. If you are working specifically with tree-based models (random forests, gradient boosting), prefer `shap.TreeExplainer` directly, as it is much faster and produces exact values for supported configurations. Reserve `KernelExplainer` for models that no other explainer supports.

6.4 Computing SHAP Values

Listing 1 shows the basic workflow for computing and visualizing SHAP values. To keep the example simple and to avoid ambiguity about output dimensionality, we use a regression model here.

```python
import shap
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# Step 1: Train a regression model
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 2: Create the SHAP explainer
# shap.Explainer auto-selects TreeExplainer for Random Forests
```

```python
explainer = shap.Explainer(model, X_train)

# Step 3: Compute SHAP values for the test set
shap_values = explainer(X_test)

# shap_values.values      -> array of shape (n_samples, n_features)
# shap_values.base_values -> array of baseline values phi_0
# shap_values.data        -> the actual feature values of X_test

# Inspect shape before interpreting
print(shap_values.values.shape)
```

Listing 1: Basic SHAP workflow using a regression model.

The object `shap_values` is an `Explanation` object that stores:

- `shap_values.values`: a matrix of SHAP values. For regression, this has shape (m, n), where entry (j, i) is the SHAP value ϕ_i for instance j. For classifiers, the shape may include an additional class dimension (see note below).
- `shap_values.base_values`: the baseline ϕ_0 (constant for all instances when using `TreeExplainer` on a fixed background).
- `shap_values.data`: the actual feature values, stored for use in visualization.

SHAP Values for Classifiers

For classifiers, the shape and interpretation of `shap_values.values` depend on the model and explainer. In binary classification, SHAP values may explain the raw model score, the log-odds, or the predicted probability, depending on the model family and the `model_output` configuration. For multi-class classifiers, SHAP values are usually computed separately for each class, often yielding an array with dimensions corresponding to samples, features, and classes.
Before interpreting signs and magnitudes, always inspect `shap_values.values.shape` and verify what the selected output represents. A positive SHAP value for a feature means "this feature increases the selected output"; what that implies for class probabilities depends entirely on which output was chosen.

6.5 Visualization

The SHAP library provides several visualization functions, each suited to a different level of analysis.

Summary Plot (Global Importance)

The summary plot shows the distribution of SHAP values for each feature across all instances in the dataset. It provides a global overview of which features matter most and in which direction.

Listing 2 illustrates two complementary ways of summarizing SHAP values across many instances. The beeswarm plot represents each instance as a point for each feature, making

it possible to inspect not only global feature importance, but also the distribution, direction, and variability of feature effects. The bar plot provides a more compact ranking by computing the mean absolute SHAP value of each feature, highlighting which variables tend to contribute most strongly to the model's predictions overall.

```
# Beeswarm plot: one dot per instance, color = feature value
shap.plots.beeswarm(shap_values)

# Bar plot: mean absolute SHAP value per feature
shap.plots.bar(shap_values)
```

Listing 2: Summary plot for global feature importance.

In the beeswarm plot, each dot represents one instance. The horizontal position shows the SHAP value (positive = feature increased the selected output; negative = feature decreased it). The color encodes the feature's actual value (in the library's default convention: red = high, blue = low). Features are sorted vertically by their mean absolute SHAP value, with the most impactful feature at the top.

Waterfall Plot (Single-Instance Explanation)

The waterfall plot explains one specific prediction. It shows the baseline, then adds each feature's contribution step by step until reaching the final prediction.

Listing 3 shows how to generate a *waterfall plot* for a single prediction using SHAP. In this example, the expression `shap_values[0]` selects the explanation associated with the first instance in the test set. The resulting plot decomposes the model output into individual feature contributions, showing how each feature pushes the prediction away from the baseline value and toward the final predicted value.

```
# Explain the first test instance
shap.plots.waterfall(shap_values[0])
```

Listing 3: Waterfall plot for a single prediction.

The waterfall plot directly visualizes the decomposition $h(x) = \phi_0 + \sum_i \phi_i$: each bar represents a ϕ_i, and the bars stack from ϕ_0 to $h(x)$. In the library's default color scheme, bars that increase the output are shown in red/pink, and bars that decrease it are shown in blue. This is the opposite of the schematic convention used in the figures of these notes; always read the direction of the bar, not just its color.

Force Plot (Compact Single-Instance View)

The force plot is a compact alternative to the waterfall plot, presenting all contributions along a single horizontal axis. It is particularly useful for embedding explanations in dashboards or reports.

Listing 4 shows how to generate a *force plot* for a single prediction. As in the waterfall plot, the expression `shap_values[0]` selects the explanation for the first test instance. The force plot provides a compact visual summary of how the feature contributions act

together, indicating which features push the prediction upward or downward relative to the baseline value.

```
# Compact force plot for the first test instance
shap.plots.force(shap_values[0])
```

Listing 4: Force plot for a single prediction.

Dependence Plot (Feature Interactions)

The dependence plot shows how a single feature's SHAP value varies as a function of its own value, and can optionally color the points by a second feature to reveal interaction effects.

Listing 5 shows how to generate a dependence plot for a single feature. In this example, `shap_values[:, 0]` selects the SHAP values associated with feature index 0 across all instances. The resulting plot helps analyze how the value of that feature is related to its contribution to the model prediction. The argument `color=shap_values` adds contextual information from the remaining features, making it easier to detect possible interaction effects or patterns that would not be visible from a purely global importance ranking.

```
# Dependence plot for feature index 0
shap.plots.scatter(shap_values[:, 0], color=shap_values)
```

Listing 5: Dependence plot to study a single feature.

Table 1 summarizes the four main plot types and their use cases.

Table 1: SHAP visualization functions and their use cases.

Plot	**Function**	**Use case**
Beeswarm / Bar	`shap.plots.beeswarm` `shap.plots.bar`	Global importance: which features matter most overall?
Waterfall	`shap.plots.waterfall`	Local explanation: what drove this specific prediction?
Force	`shap.plots.force`	Local explanation: compact single-instance view
Scatter / Dependence	`shap.plots.scatter`	Feature behavior: how does a feature's effect change with its value?

Listing 6 illustrates the explicit use of `TreeExplainer` for a tree-based model. The example first trains an XGBoost classifier on the breast cancer dataset and then builds a SHAP explainer specialized for tree ensembles. This is useful because tree-based models admit particularly efficient SHAP computations. After computing the explanations with `explainer(X)`, the code inspects the shape of the resulting SHAP values before producing visualizations. This step is important because the structure of the output may

vary depending on the task, the model, and the library version, especially in classification problems. The final commands generate a global summary plot and a local waterfall plot, connecting the same set of SHAP values to both model-level and instance-level interpretation.

```python
import shap
import xgboost as xgb
from sklearn.datasets import load_breast_cancer

# Load data and train an XGBoost model
X, y = load_breast_cancer(return_X_y=True, as_frame=True)
model = xgb.XGBClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

# TreeExplainer: efficient and exact SHAP values for tree-based
    models
explainer = shap.TreeExplainer(model)
shap_values = explainer(X)

# Always inspect shape before interpreting
# For binary classifiers, shape may be (n_samples, n_features, 2)
# depending on the version and model configuration
print(shap_values.values.shape)

# Summary plot: global feature importance
shap.plots.beeswarm(shap_values)

# Waterfall plot: single instance
shap.plots.waterfall(shap_values[0])
```

Listing 6: Using TreeExplainer explicitly for tree-based models.

Entry Ticket 5

A data scientist uses `KernelExplainer` on a neural network model with 100 input features and a background dataset of 500 instances.

1. Why can't `TreeExplainer` be used here? What does `KernelExplainer` require to compute SHAP values?
2. `KernelExplainer` works by fitting a weighted linear regression to approximate SHAP values. In your own words, why does this approximation converge to the true Shapley values as the number of samples increases?
3. The computation is very slow. Suggest two ways to speed it up without changing the model type.
4. After computing SHAP values, the data scientist observes that one feature has SHAP values of zero for every instance. What does this mean about the feature in the model?

7 Interpreting SHAP Values

7.1 Local vs. Global Explanations

SHAP values are inherently local: each ϕ_i describes the contribution of feature i to one specific prediction. To obtain global insights, we aggregate SHAP values across many instances. The most common global measure is the mean absolute SHAP value:

$$\bar{\phi}_i = \frac{1}{m} \sum_{j=1}^{m} |\phi_i^{(j)}|,$$

where $\phi_i^{(j)}$ is the SHAP value of feature i for instance j. This quantity measures, on average, how much feature i moves the selected model output away from the baseline, and can be used as a global feature importance score.

SHAP Global Importance vs. Built-in Feature Importance

Tree-based models typically provide a built-in feature importance measure based on split counts or impurity reduction. These measures are fast to compute but can be unreliable: they tend to favor features with many possible split points (such as continuous features with many unique values) and do not account for feature interactions. SHAP-based global importance is more consistent and robust, because it is rooted in the game-theoretic axioms.

7.2 Reading a Waterfall Plot

The waterfall plot is the primary tool for understanding a single prediction. Reading it correctly requires understanding the following conventions as implemented in the SHAP library:

- The plot starts at ϕ_0 (the baseline) at the bottom.
- Each subsequent bar represents one feature's ϕ_i. By the library's default color convention, bars that increase the selected output are shown in red/pink, and bars that decrease it are shown in blue.
- Features are sorted by the absolute magnitude of their SHAP value: the most impactful feature appears first.
- The top of the plot shows the final prediction $h(x)$, which equals $\phi_0 + \sum_i \phi_i$.

7.3 Reading a Beeswarm Plot

The beeswarm plot provides a global view of feature behavior:

- Each row corresponds to one feature, sorted by mean absolute SHAP value (most important at the top).
- Each dot represents one instance. The horizontal position is the SHAP value; the color encodes the feature's actual value (red = high, blue = low in the library's default scheme).
- A cluster of red dots with large positive SHAP values means that high values of that feature tend to increase the selected model output.

- A cluster of blue dots with large positive SHAP values means that *low* values of that feature tend to increase the selected model output (counterintuitive effects).

Common Misconception

A large positive SHAP value for feature i in one instance does *not* mean that feature i always helps. The same feature might have negative SHAP values for other instances, or for a different model. SHAP values are prediction-specific. To assess whether a feature consistently increases or decreases the selected output, examine its distribution of SHAP values across many instances, for example through a beeswarm or dependence plot.

8 Final Remarks

8.1 Summary of Key Concepts

The following points summarize the essential content of these notes:

- **SHAP values** are feature attribution scores grounded in cooperative game theory (Shapley values). They explain individual predictions of any machine learning model.
- The **baseline** ϕ_0 is the model's expected prediction over a background dataset. It represents the average prediction over the reference population, and SHAP values measure deviations from this reference.
- The **marginal contribution** Δ_i^S measures how much feature i changes the expected model output when it is added to a coalition S of already-known features, under the interventional definition.
- The **SHAP value** ϕ_i for feature i is the weighted average of its marginal contribution Δ_i^S across all possible subsets of the remaining features.
- SHAP values satisfy the **decomposition property**: $h(x) = \phi_0 + \sum_i \phi_i$. The prediction is exactly accounted for by the baseline and the feature contributions (exactly for analytic methods; up to small numerical error for approximate ones).
- The four properties of **local accuracy, missingness, consistency, and symmetry** uniquely characterize SHAP values among all additive explanation methods.
- **Exact SHAP computation** has exponential complexity ($\mathcal{O}(2^n)$). The SHAP Python library provides efficient approximations, including exact polynomial-time algorithms for tree-based models (`TreeExplainer`).
- The SHAP library provides **visualization tools** (beeswarm, waterfall, force, and dependence plots) for both global and local interpretation.
- SHAP values explain the **model's output**, not the real-world phenomenon. They are associational, not causal.

8.2 Common Pitfalls

- **Confusing local and global explanations.** A SHAP value explains one prediction, not the model's general behavior. Do not conclude that a feature is globally unimportant because it has a small SHAP value on one instance.
- **Ignoring the background dataset.** SHAP values depend on the background dataset. If the background is unrepresentative, the explanations may be misleading. Always use a background that reflects the intended input distribution.
- **Using the wrong explainer.** Applying `KernelExplainer` to a tree model is valid but unnecessarily slow. Applying `TreeExplainer` to a neural network will fail. Always match the explainer to the model type.
- **Not checking what output is being explained.** Before interpreting SHAP values for a classifier, verify whether the explainer is decomposing probabilities, log-odds, or raw scores. The sign and magnitude of ϕ_i are only interpretable relative to the chosen output scale.
- **Ignoring correlated features.** When two features carry similar information, SHAP may split attribution between them or assign importance in a way that depends on the background distribution and the feature-dependence assumption. A low SHAP value for one of two highly correlated features does not necessarily mean that the underlying information is unimportant to the model. Investigate correlated features together.
- **Interpreting SHAP values as causal.** SHAP values describe how much each feature changes the model's prediction, not how much it causally affects the real-world outcome. A feature with a large SHAP value might be correlated with the true cause, but SHAP cannot establish causation.

Practical Guidelines for SHAP

- **Explainer choice**: use `shap.Explainer` (auto) or `TreeExplainer` for tree models; use `KernelExplainer` only when necessary.
- **Background dataset**: use a random sample of 50–200 training instances; ensure it represents the marginal feature distribution.
- **Output scale**: always verify what the SHAP values are decomposing (predicted value, log-odds, probability, etc.) before interpreting.
- **Global analysis**: use beeswarm or bar plots to identify which features matter most across the dataset.
- **Local analysis**: use waterfall or force plots to explain individual predictions.
- **Interaction analysis**: use scatter/dependence plots to study how a feature's SHAP value varies with its own value.
- **Sanity check**: always verify that $\phi_0 + \sum_i \phi_i \approx h(x)$ for a few instances.
- **Causality**: remember that SHAP values are associational, not causal. Treat them as descriptions of the model, not the world.

8.3 Connection to Other Interpretability Methods

SHAP values are part of a broader landscape of model interpretability methods. Table 2 positions SHAP relative to other common approaches.

Table 2: Comparison of common model interpretability methods.

Method	**Scope**	**Model-agnostic?**	**Key idea**
SHAP	Local + Global	Yes	Shapley value attribution
LIME	Local	Yes	Local linear approximation
Permutation importance	Global	Yes	Shuffle feature; measure loss increase
Partial dependence plots	Global	Yes	Marginal effect of one feature
Attention weights	Local	No (Transformers)	Internal model attention
Surrogate decision tree	Global	Yes	Approximate with an interpretable tree trained to mimic the black-box model

SHAP is distinguished from LIME in that it satisfies the four Shapley axioms and is therefore guaranteed to be consistent and locally accurate, whereas LIME's linear approximations do not provide these guarantees. SHAP is preferred when a principled, theoretically grounded attribution is required.

Entry Ticket 6

A student trains a gradient-boosted tree to predict whether a loan applicant will default. The model achieves 92% accuracy on the test set. She then uses SHAP to explain individual predictions.

1. For applicant A, the SHAP value for "annual income" is -0.35. Before interpreting this, what should the student verify about the output being explained?
2. The beeswarm plot shows that the feature "credit history length" has the largest mean absolute SHAP value. Does this mean that increasing a customer's credit history length would reduce the probability of default? Explain why or why not.
3. She notices that the SHAP values for two features ("age" and "years of employment") are almost identical for every instance. What might explain this, and what problems could it cause for interpretation?
4. Why is it important to verify $\phi_0 + \sum_i \phi_i \approx h(x)$ before trusting the explanation?

8 References

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774. Curran Associates, 2017.

Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games, Volume II*, volume 28 of *Annals of Mathematics Studies*, pages 307–317. Princeton University Press, Princeton, NJ, 1953.